CS6008 Assignment 2
DNS Security

Arun P Madhu (25M2017) Naveen A (25M1111)

October 2026

Repository: https://github.com/arunser/CS6008_Assignment_2

1 Overview

This report covers an isolated DNS hierarchy that we built in Docker and then attacked and defended one property at a time. The idea of the assignment is the same one we found useful in the first assignment: you understand a protection much better once you have broken the version without it yourself. So from Phase 2 onward we wrote a small attack tool, watched it work, added the matching defence, and then ran the same attack again to show the defence holding.

The hierarchy has three subnets that are joined only by two router containers, with no NAT and only static routes. The client subnet `172.16.1.0/24` holds the client, the real web server at `172.16.1.20` and a rogue web server at `172.16.1.30`. The resolver subnet `172.16.2.0/24` holds a recursive BIND resolver at `172.16.2.10`. The authoritative subnet `172.16.3.0/24` holds the root server, the `com` server and the `example.com` server. The edge router joins the client and resolver subnets, and the core router joins the resolver and authoritative subnets. Because those two routers are the only way across, we can place a single attacker on the path and study exactly what it can and cannot do.

Every attack tool is written in C with libpcap, which does both the packet capture and the raw injection. We did not use Scapy, dnspython or any other high level library for the attacks, as the assignment requires. The DNSSEC keys and the TLS certificates are generated at deploy time into Docker volumes, so no private key material is ever committed to the repository. Each phase lives in its own directory with a short README, the exact commands we ran, and the screenshots we captured under `evidence`.

2 Phase 1: The plaintext baseline

We brought up the full hierarchy and checked that it actually resolves. The client asks the resolver for `example.com` and gets back `172.16.1.20`, and `curl` returns the real page, which confirms the recursive path works end to end. We also checked isolation: a direct query from the client straight to the authoritative server times out, because the only route to the authoritative subnet runs through the routers and not directly from the client.

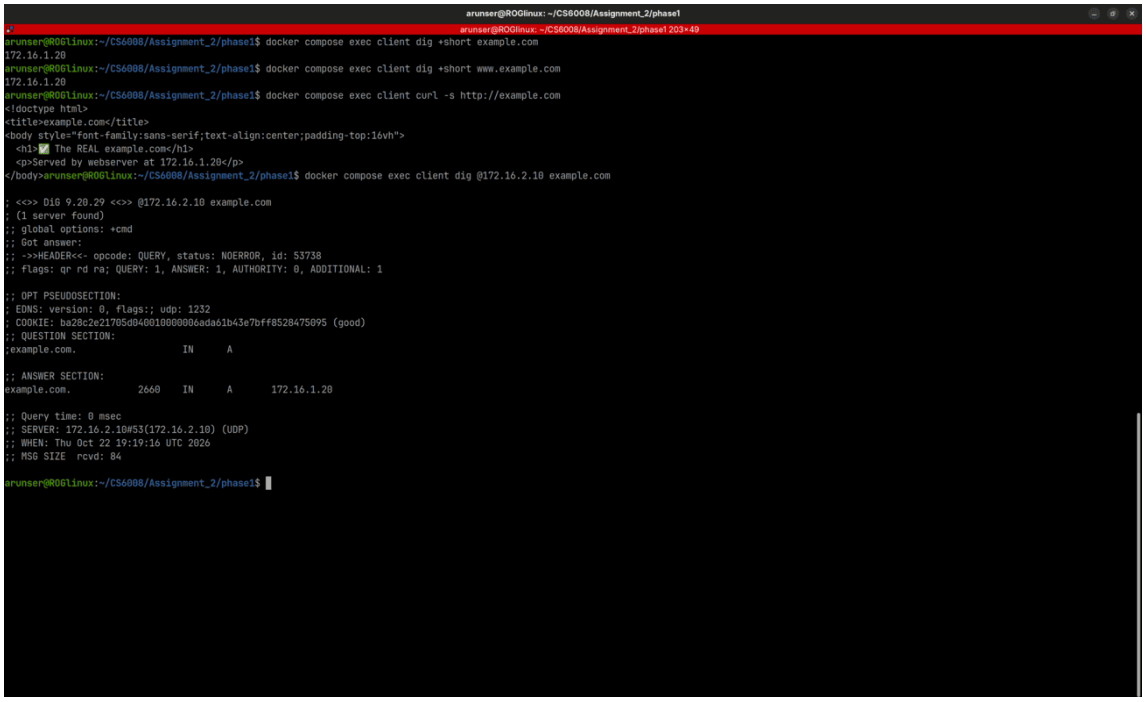


Figure 1: The client resolves example.com to 172.16.1.20 and fetches the real page.

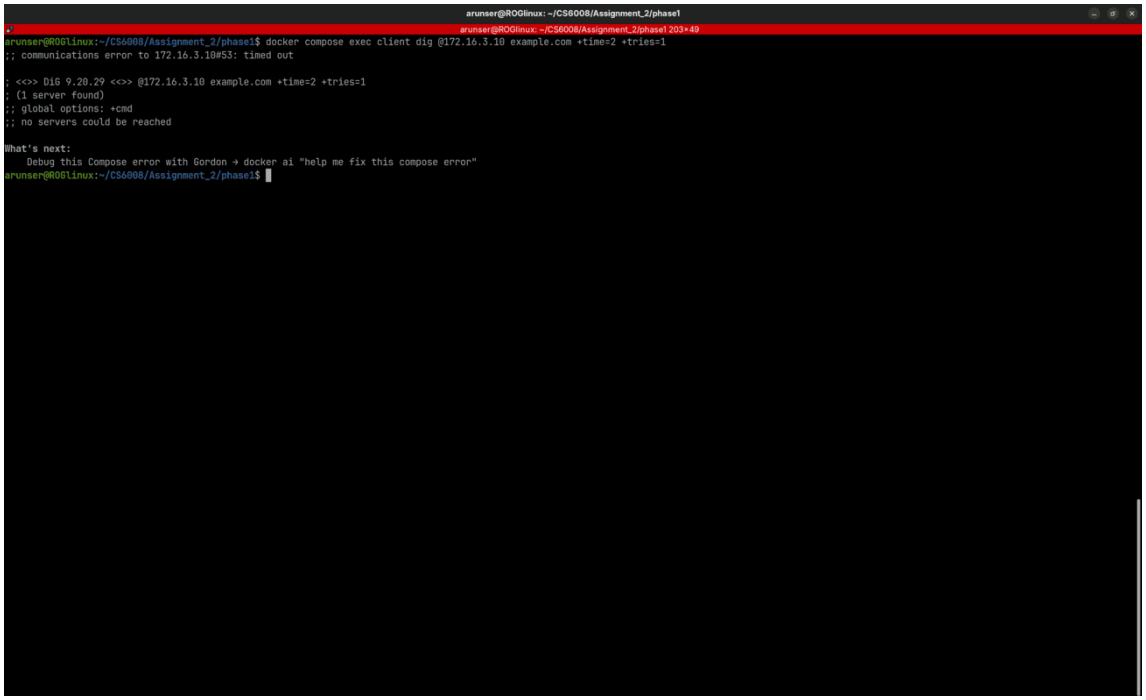


Figure 2: A direct query to the authoritative subnet times out. The client can only reach it through the resolver and the routers.

The assignment asks us to trace the referral chain from the capture. In Wireshark we can see the resolver walk the hierarchy from the top. It asks the root server for `example.com`, the root refers

it to the `com` servers, it asks `com`, `com` refers it to the `example.com` server, and that server finally returns the A record. The important observation is that every one of these messages is in the clear. The query names, the sixteen bit transaction identifiers and the answers are all plainly visible on the wire, with nothing signed and nothing encrypted.

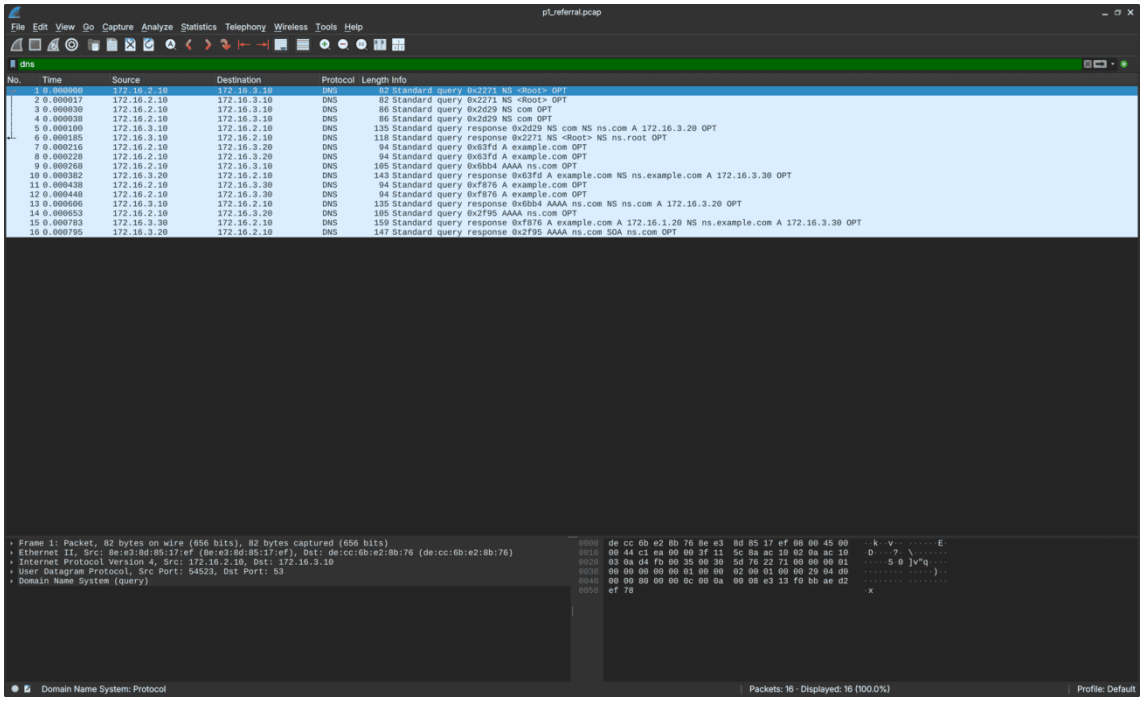


Figure 3: The Root to TLD to Domain referral chain in Wireshark, entirely in cleartext.

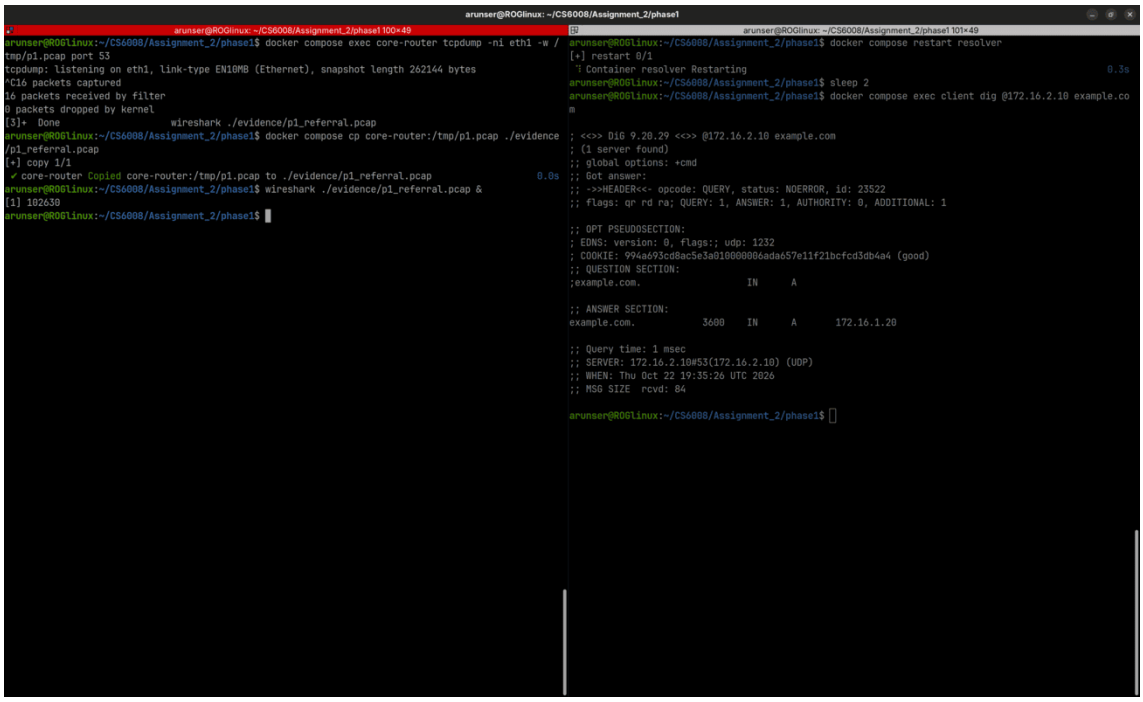


Figure 4: The commands used to produce the referral capture.

For the routing question, the resolver reaches the root server using a static route for the authoritative subnet that points at the core router, so its queries cross the resolver subnet, enter the core router, and arrive on the authoritative subnet. There is no default gateway anywhere, which is what keeps the whole thing self contained.

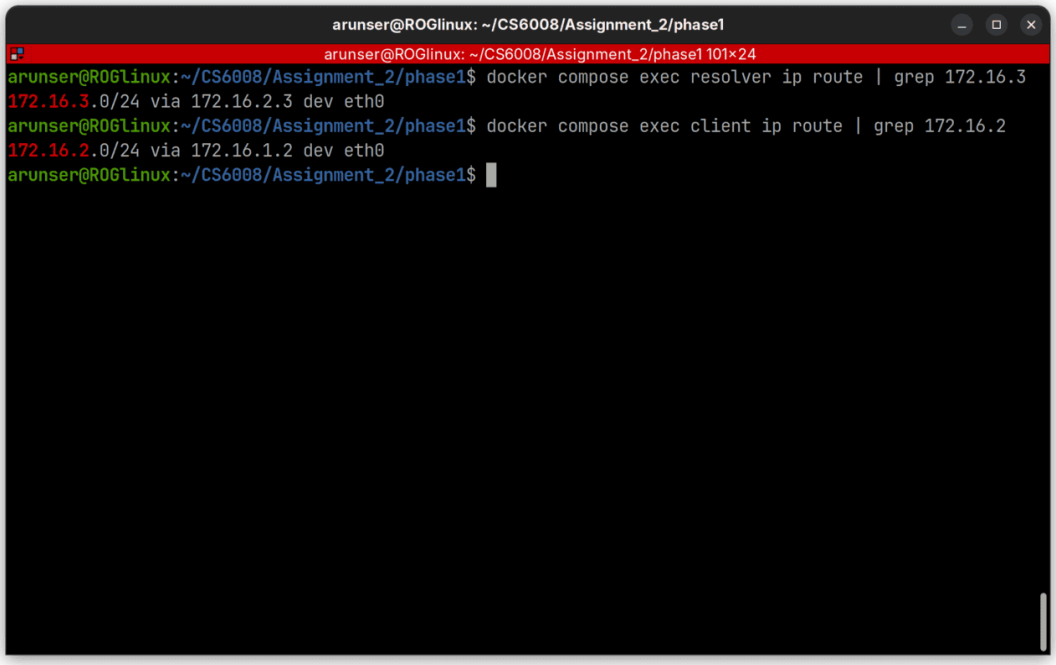


Figure 5: The static routes that carry the resolver's queries into the authoritative subnet.

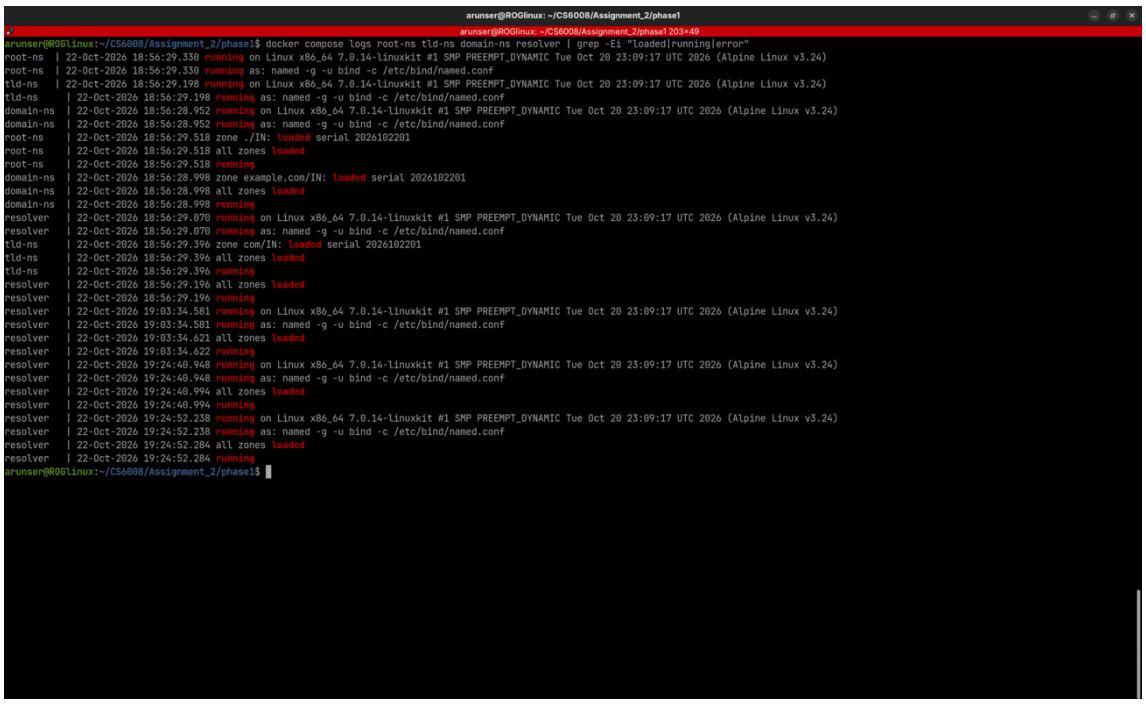


Figure 6: Each authoritative server has loaded its zone.

The last question asks what a node on the resolver's path could do to this resolution. Because the traffic is cleartext and nothing is authenticated, such a node can read every query and, far more

dangerously, it can rewrite the answers before they get back. That is exactly the capability we build and use in the next phase. The lesson of Phase 1 is simply that plain DNS trusts whatever arrives, and on this topology the path is a real place an attacker can sit.

3 Phase 2: On-path response forgery

We added an attacker on the resolver subnet running a small libpcap interceptor, a rogue web server, and a route on the core router that sends the resolver's inbound replies through the attacker. With forging turned off the interceptor just relays everything, so resolution still returns the real server, which we verified first so we knew the relay itself was transparent.

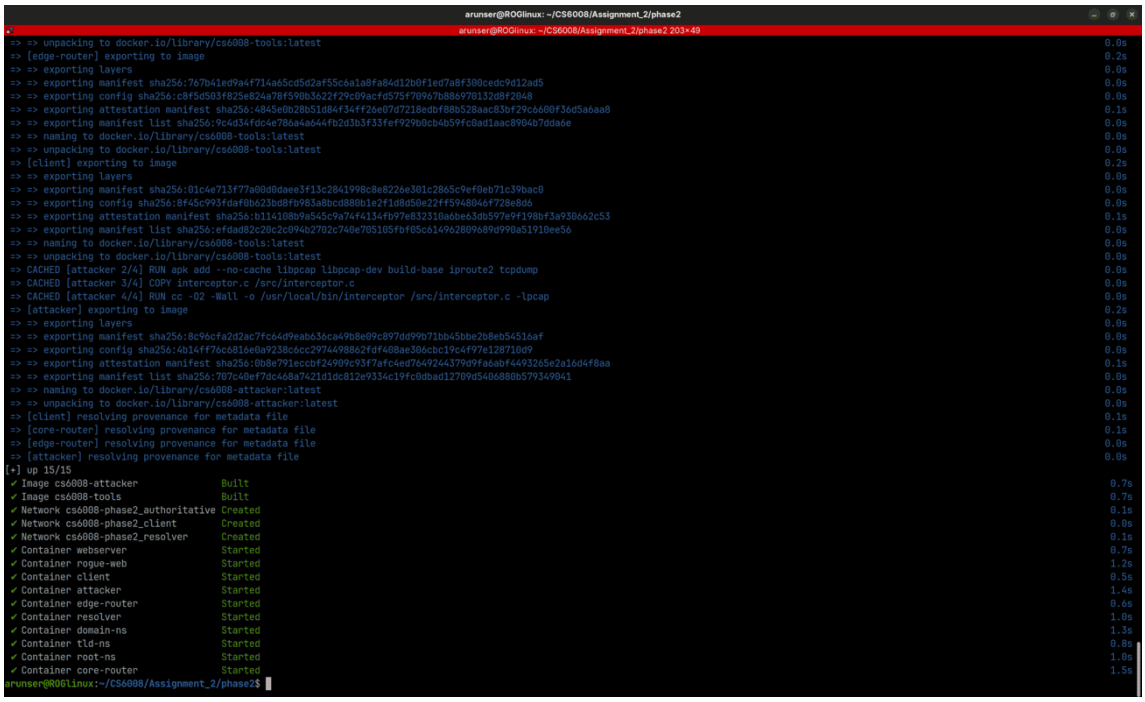


Figure 7: The Phase 2 stack with the attacker and the rogue server.

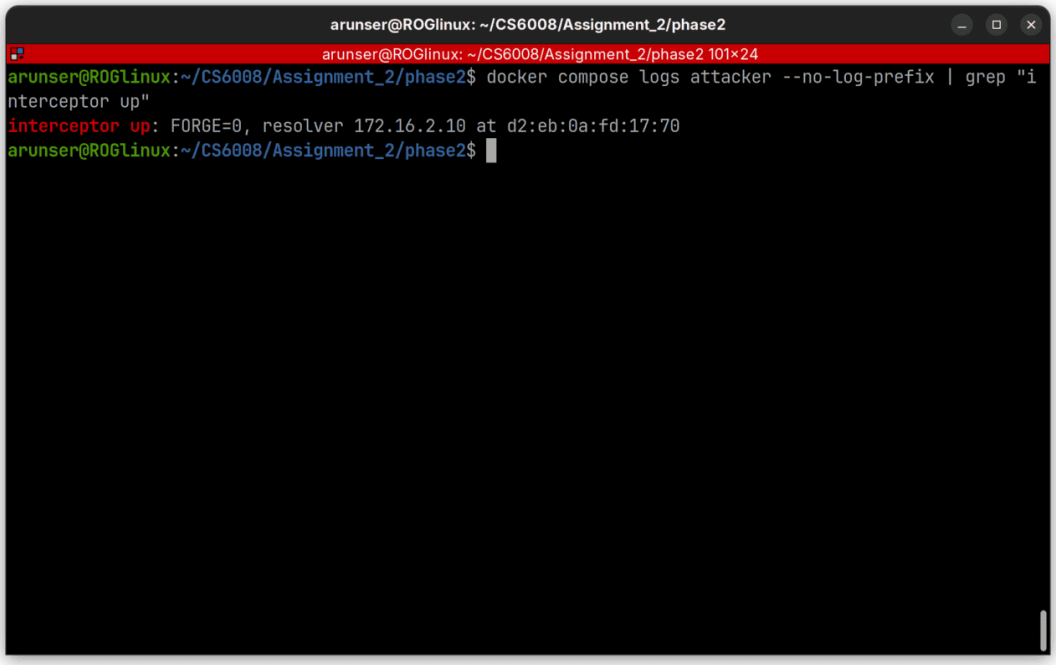


Figure 8: The interceptor has learned the resolver's MAC and is ready to relay.

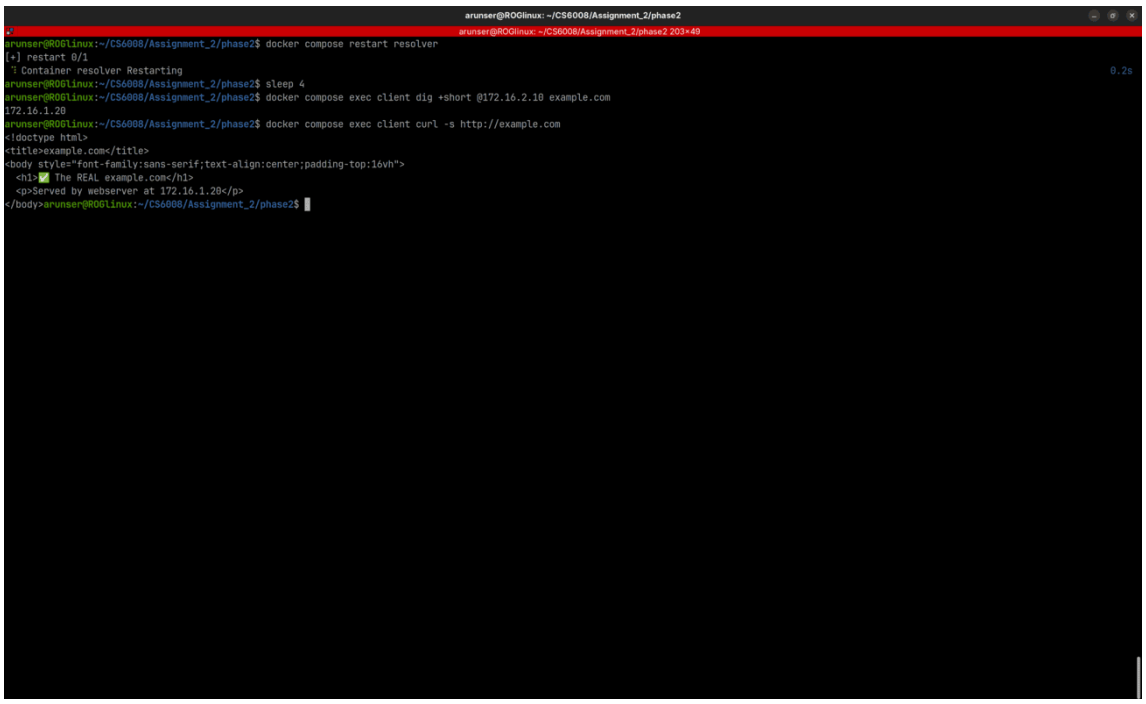


Figure 9: With forging off, resolution returns the real server at 172.16.1.20.

With forging turned on, the resolver caches `example.com` pointing at `172.16.1.30` and the client's fetch returns the rogue page, while every other name keeps resolving normally. The attacker also

prints a line for each hijacked lookup, and in the capture we can follow the genuine answer coming in from the domain server and the forged one going back out with the same transaction identifier.

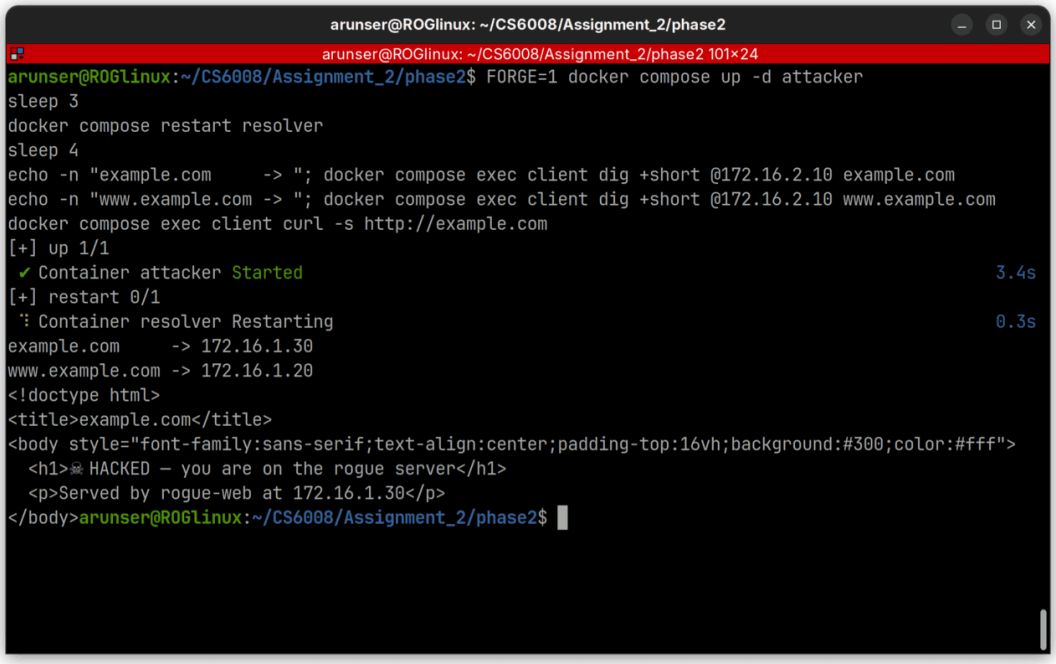


Figure 10: With forging on, example.com resolves to the rogue server and www.example.com is left untouched.

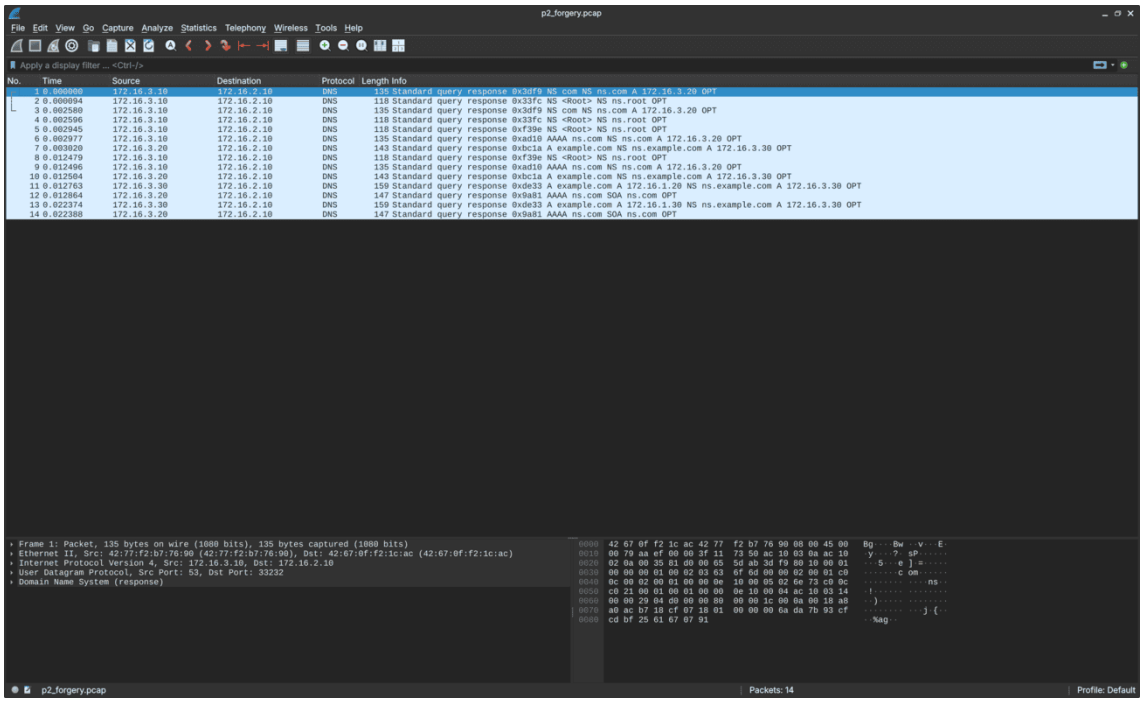


Figure 11: The genuine answer and the injected answer carrying the same query identifier.

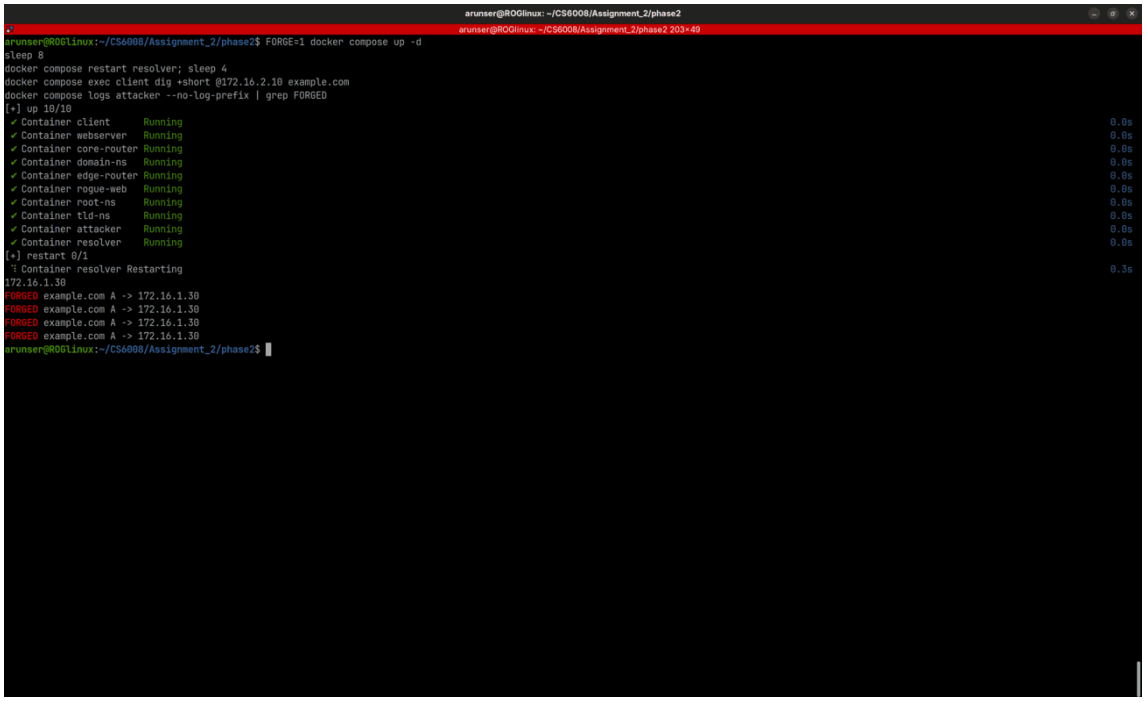


Figure 12: The attacker's log line for the hijacked lookup.

The assignment asks why the attack succeeds and what our on-path position buys us. Plain DNS gives the resolver no way to tell a real answer from a forged one. It accepts any well formed reply that matches the question and the transaction identifier. Sitting on the path means we can read the real query and the real answer, so we never have to guess the identifier or win a race against the real server. That is the whole difference from an off path attacker, who cannot see the query and has to brute force the identifier and the source port while racing the genuine reply. We get to simply rewrite the real response as it passes.

It also asks how the relay leaves other names alone. The interceptor parses each DNS response and only acts when the question is `example.com` of type A. In that one case it overwrites the four byte address in the answer and recomputes the IP and UDP checksums. Everything else is relayed byte for byte, which is why `www.example.com` and every other name stay correct.

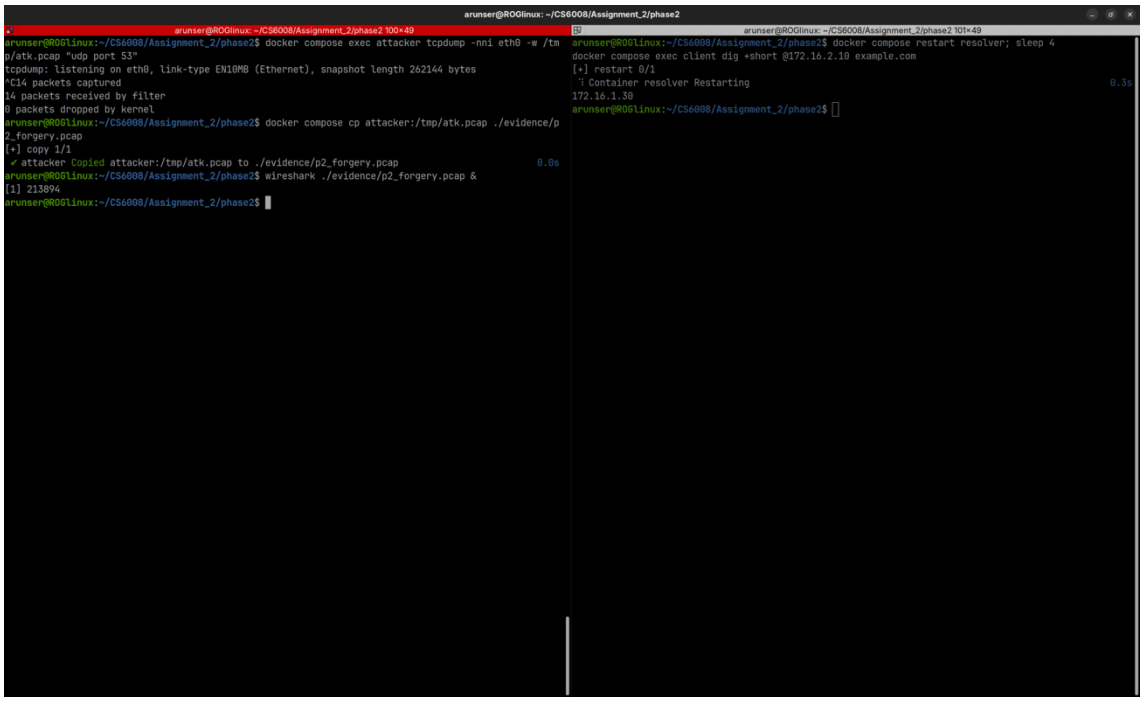


Figure 13: The commands used to capture the forged exchange.

The one assumption worth stating is that the attacker is genuinely on the path, which the topology gives us through the diversion route on the core router rather than through any weakness we had to invent. Given that position, Phase 2 shows that plaintext DNS offers no protection at all against someone who can see and edit the traffic.

4 Phase 3: Defending with DNSSEC

In Phase 3 we left the attacker exactly as it was and changed only the data. Every zone is now DNSSEC signed and the resolver validates. Each authoritative server signs its own zone and hands its delegation signer up to its parent, so the chain runs from the root down to `example.com`, and the resolver pins the root key as a trust anchor. We first confirmed the healthy case: a lookup comes back with the authenticated data flag set and a signature beside the address, and a locally validating client reports the chain as fully validated.

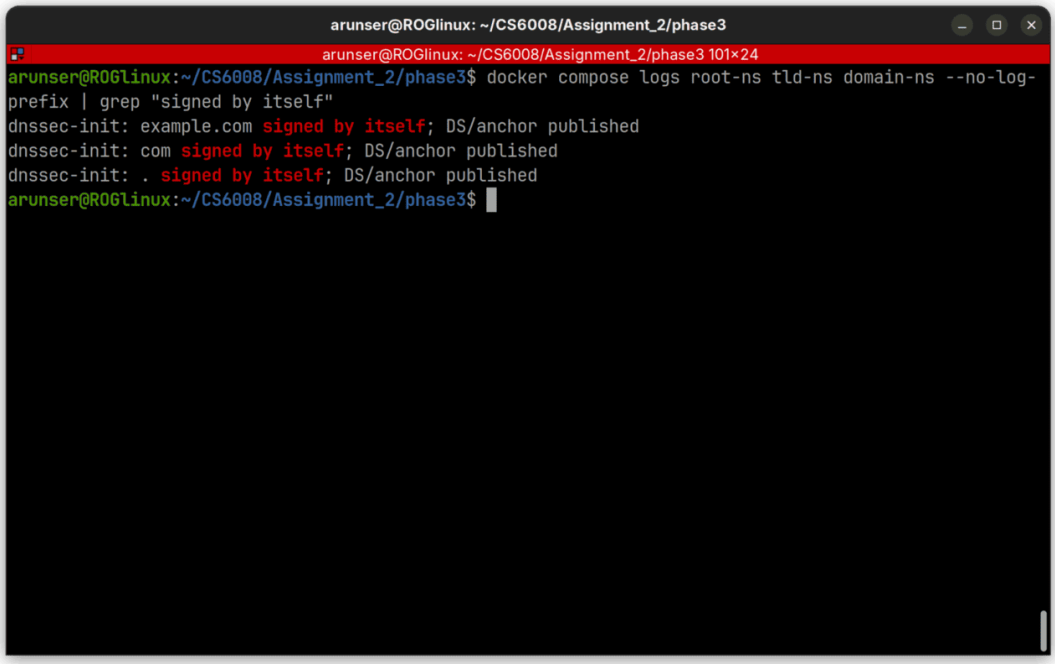


Figure 14: Each zone signs itself and publishes its delegation signer upward.

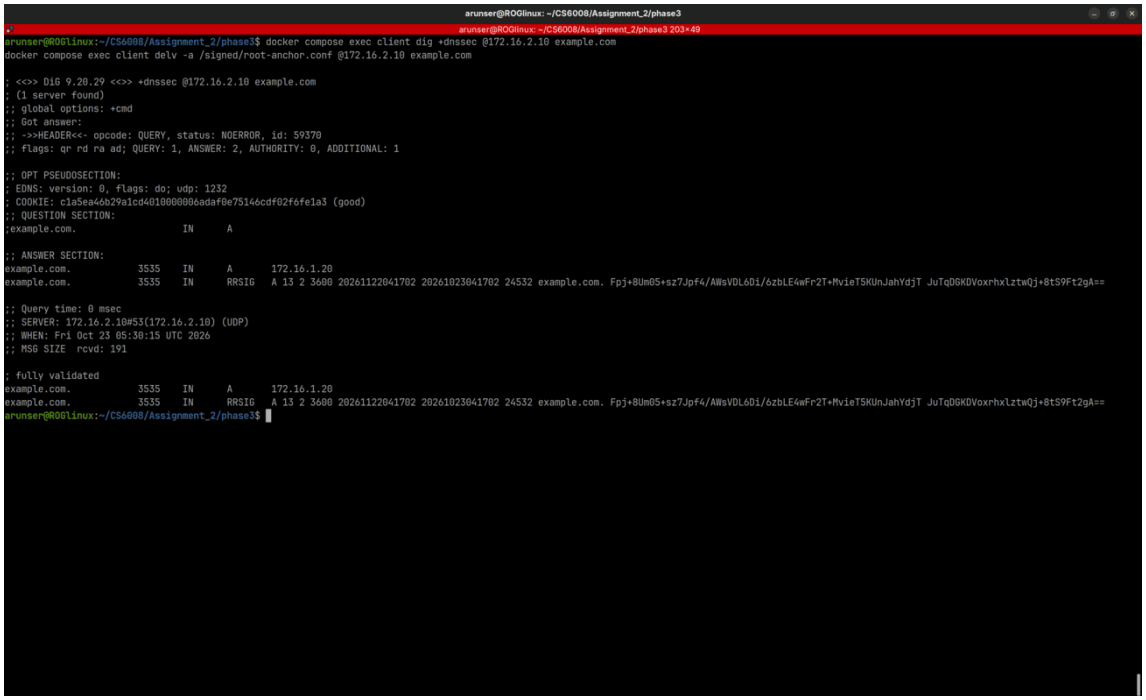


Figure 15: A healthy lookup carries the authenticated data flag and a valid signature, and the client validates the whole chain.

Then we ran the same forgery from Phase 2. This time the forged address no longer matches its signature, so the resolver refuses it and returns SERVFAIL, and the client's fetch fails with no rogue

page. The resolver's log says the signature failed to verify, while the attacker's log shows it is still forging. The attack is unchanged and still active, but the validating resolver catches the mismatch and fails closed.

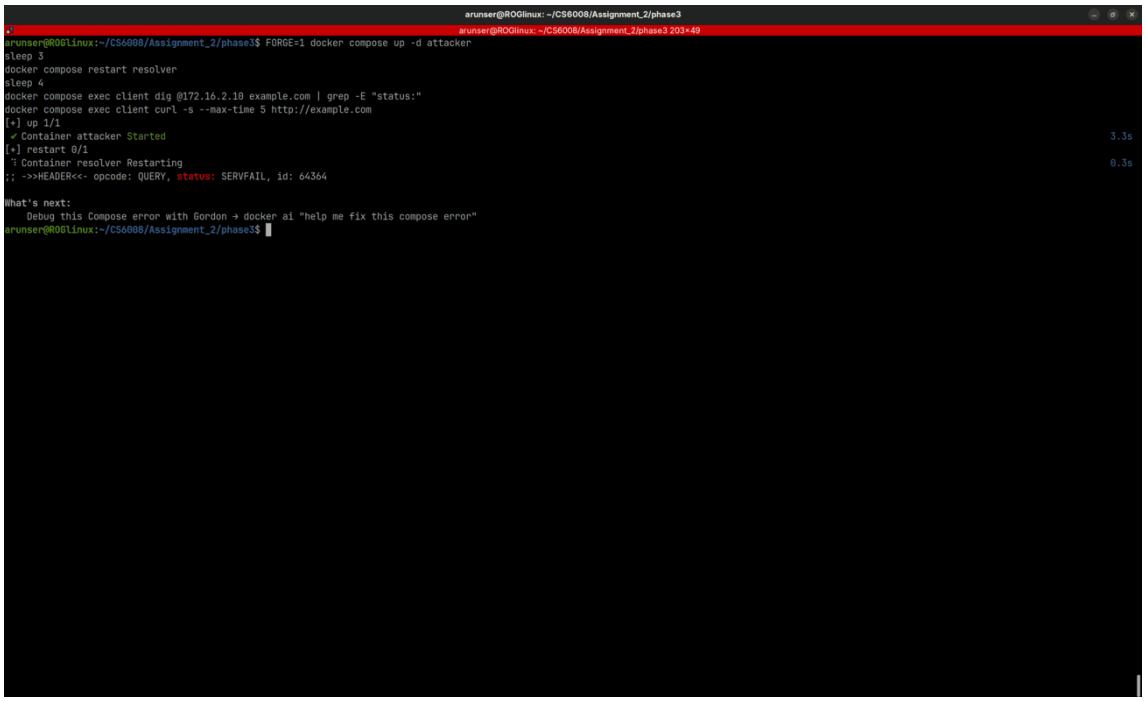


Figure 16: The same forgery now returns SERVFAIL instead of the rogue page.

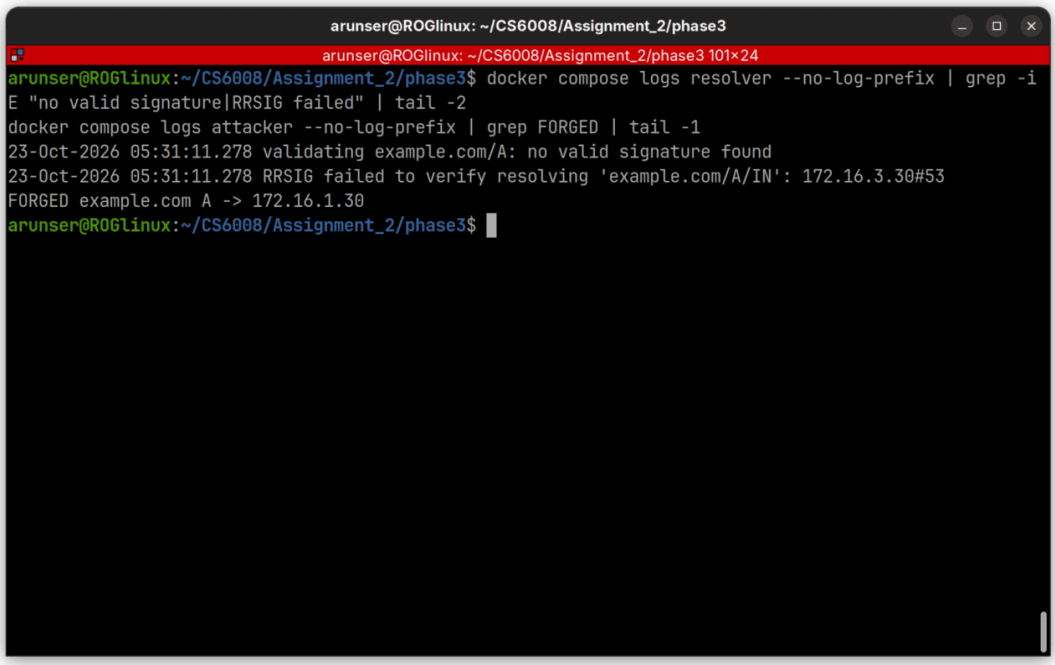


Figure 17: The resolver logs the signature failure while the attacker logs that it is still forging.

This is the direct contrast the assignment asks for. In Phase 2 the client ended up on the rogue page; here the client gets nothing, because the resolver now checks the address set against its signature and trusts the key through the delegation chain from the root anchor. Comparing the wire with Phase 1, the on-path attacker can still read everything. The traffic is the same cleartext, only now carrying extra signature, key and delegation records. DNSSEC protects integrity and authenticity, so tampering is detected, but it does nothing for confidentiality, since nothing is encrypted. That gap is the whole reason Phase 7 exists.

On the signing choice, we used ECDSA with the P-256 curve and SHA-256, which is algorithm number thirteen. It gives small keys and signatures and fast validation, against RSA's larger and slower keys, and it is universally supported now, so it is the sensible modern default. The keys are generated at deploy into a shared volume, and only the public trust anchor is referenced, so no private key sits in the repository.

5 Phase 4: Signature stripping and downgrade

Forging a signed answer failed, so in Phase 4 the attacker stops trying to forge a signed record and instead tries to make the signed zone look unsigned. We added a strip mode to the interceptor that, for `example.com`, rebuilds the answer as a single unsigned forged address with no signature, and returns an empty answer for the key and, in the fuller version, the delegation signer records. We verified first that before stripping the name still validates.

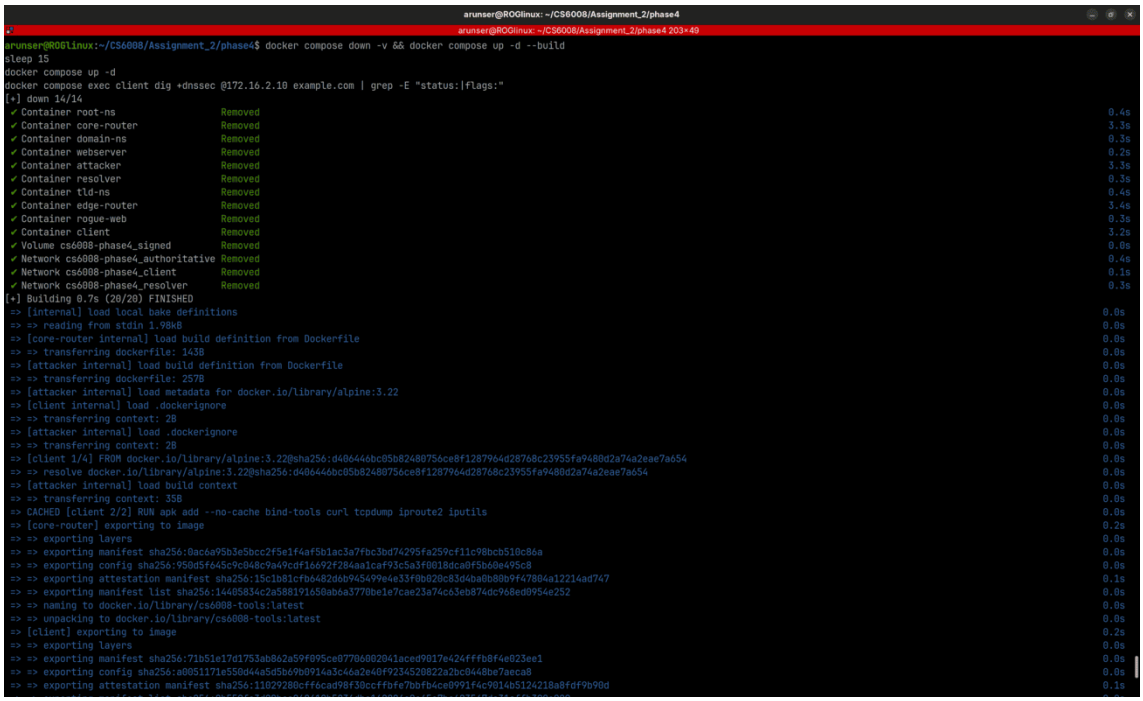


Figure 18: The signed hierarchy deployed for Phase 4.

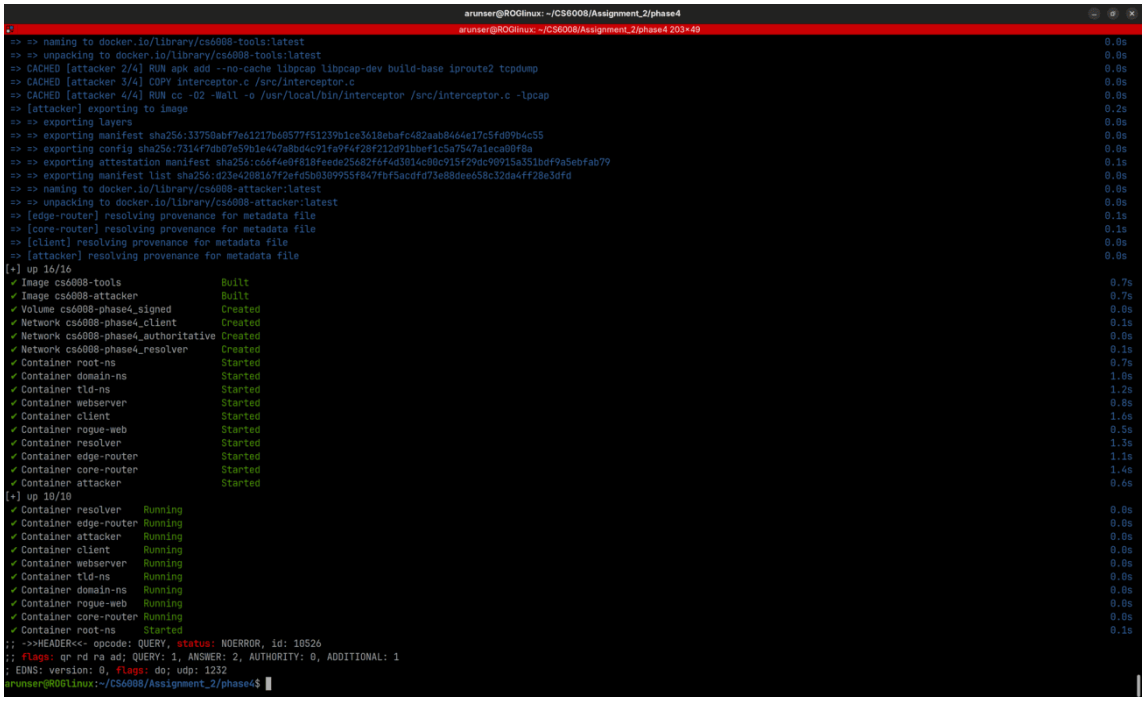


Figure 19: Before stripping, example.com still resolves as an authenticated answer.

In the first run the chain is left intact. We strip the signature and the key, and in the fuller downgrade we also strip the delegation signer and its signature from the `com` referral, and inject the forged address. Both attempts still come back as SERVFAIL, and the resolver logs that it could not build an insecurity proof. The reason is that `com` holds a signed delegation signer for `example.com`. The parent vouches, on a channel the attacker cannot forge, that the child is signed, so the resolver knows signatures are required and refuses the stripped answer. You cannot strip your way out of a zone the parent has already vouched for.

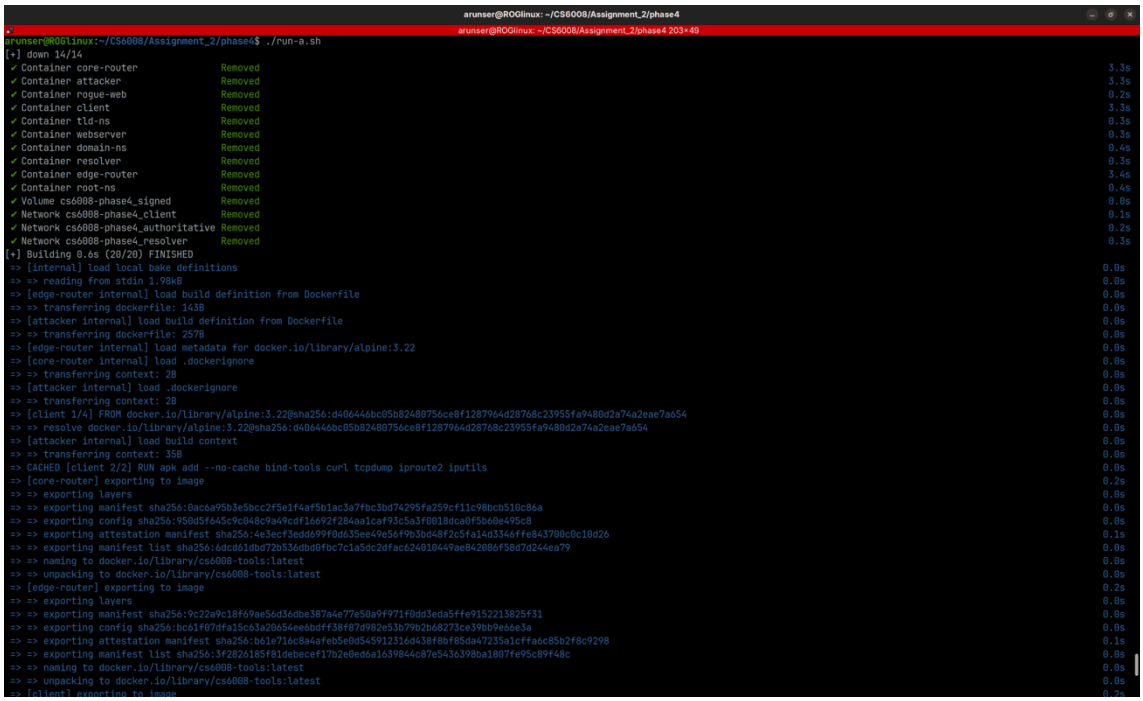


Figure 20: Run A drives the strip attack from a script.

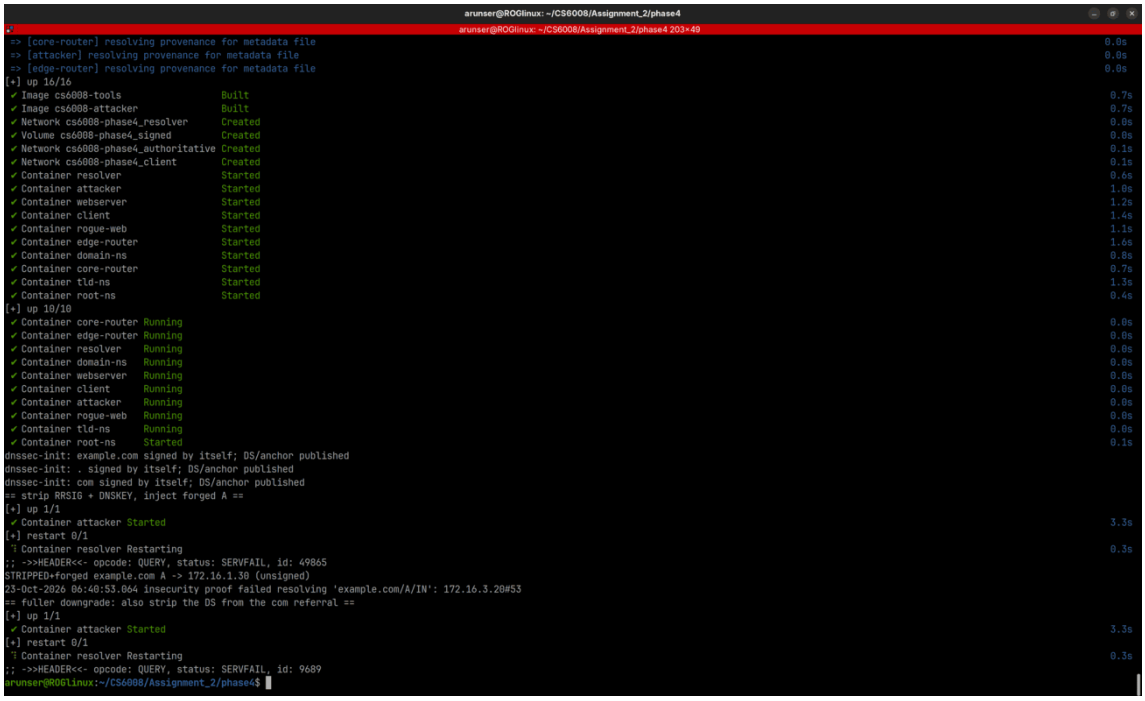


Figure 21: Run A: both the plain strip and the fuller strip return SERVFAIL, with the insecurity proof failing.

In the second run we changed exactly one thing. We re-signed so that `com` publishes no delegation signer for `example.com`, which is the mistake of an operator who signs the child but never gives

the delegation signer to the parent. Now `com` authentically proves that the delegation is insecure, so the resolver stops requiring signatures, accepts the stripped forgery, and the client lands on `172.16.1.30`.

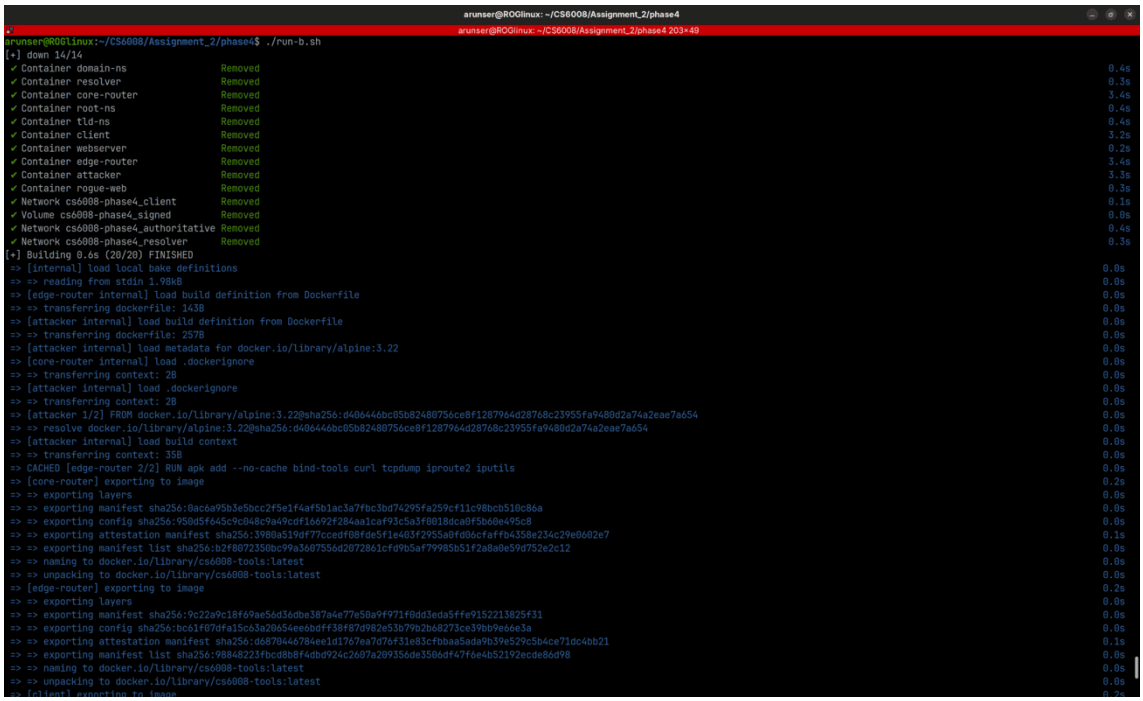


Figure 22: Run B re-signs with no delegation signer in the parent.

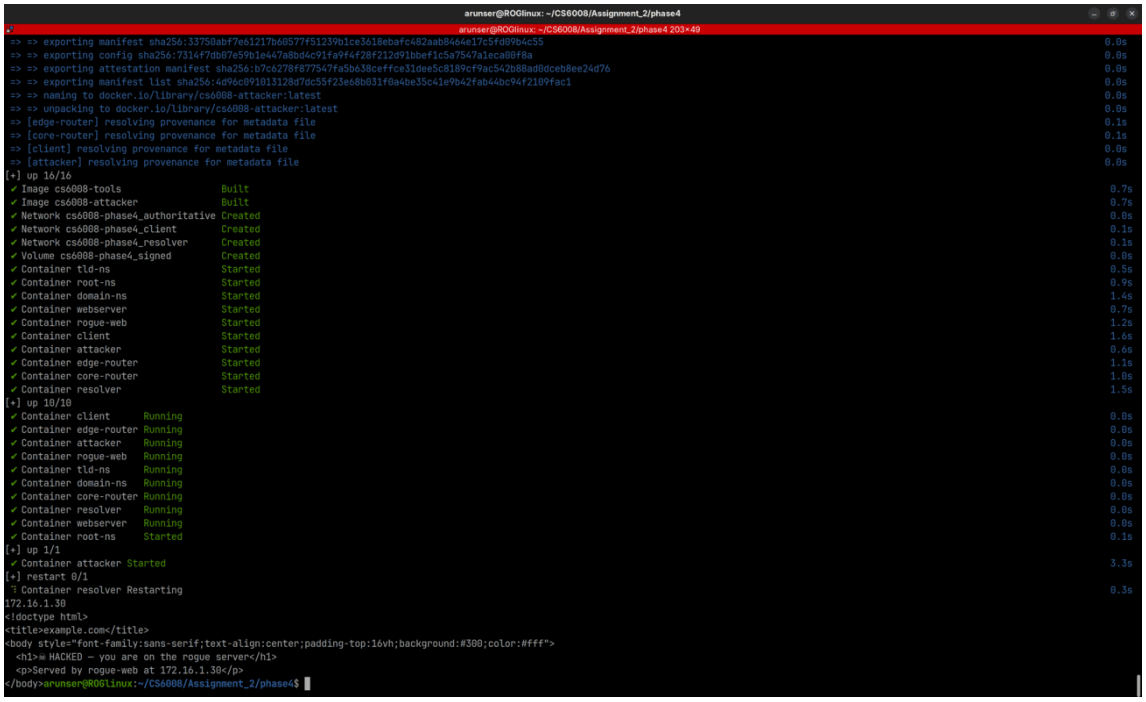


Figure 23: Run B: with the delegation signer missing, the forged answer is accepted and the client reaches the rogue page.

The contrast answers where downgrade resistance comes from. It comes entirely from the delegation signer in the parent, the one secure signed link that says the child is signed. The mistake that forfeits it is signing a zone but never publishing its delegation signer in the parent, which leaves the zone signed but insecure and therefore fully forgeable again. One missing record turns a protected zone back into an unprotected one.

6 Phase 5: NSEC zone walking

DNSSEC also has to prove that a name does not exist, and with plain NSEC it does that by pointing at the next existing name in the zone. We wrote a second libpcap tool, a walker, that abuses this. Starting from `example.com` alone it asks for a name's NSEC record, reads the next name it points at, asks for that, and keeps following the chain until it wraps back to the start. It talks to the domain server directly and uses only ordinary queries, with no zone transfer and no wordlist. First we confirmed that a denial for a missing name is itself authenticated.

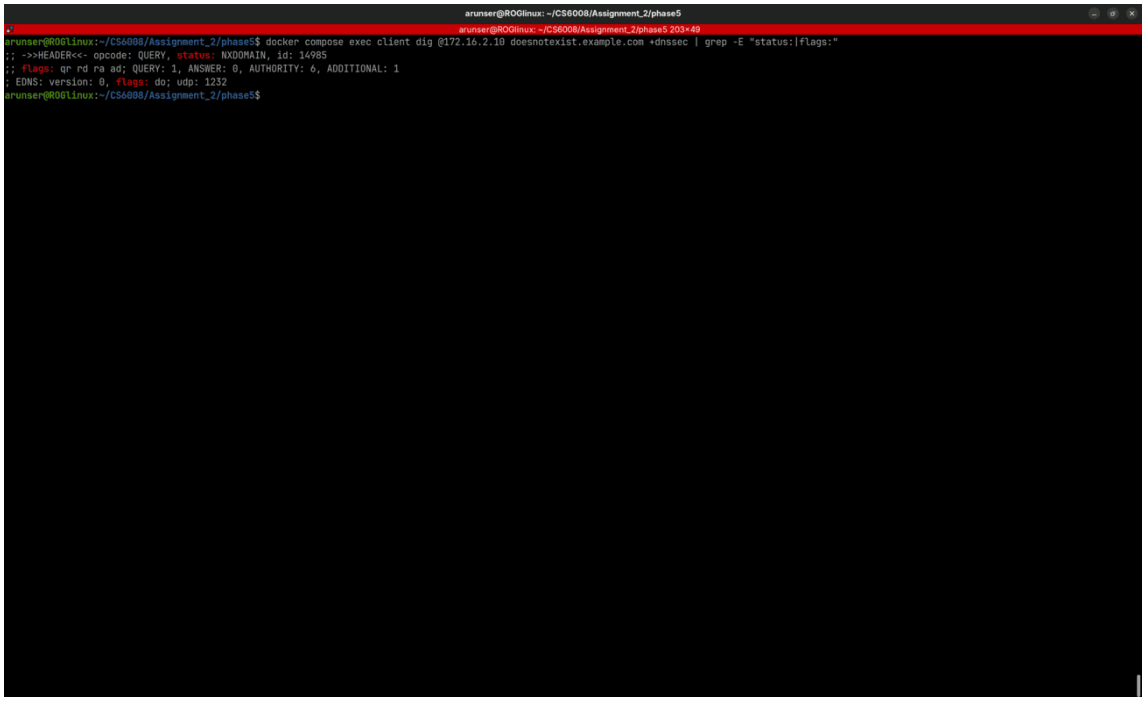


Figure 24: A missing name returns an authenticated NXDOMAIN carried by signed NSEC records.

The walker then recovered the whole zone, the names db, dev, internal, mail, ns, vpn and www under `example.com`, in eight queries, following the next name field from the start until it wrapped around.

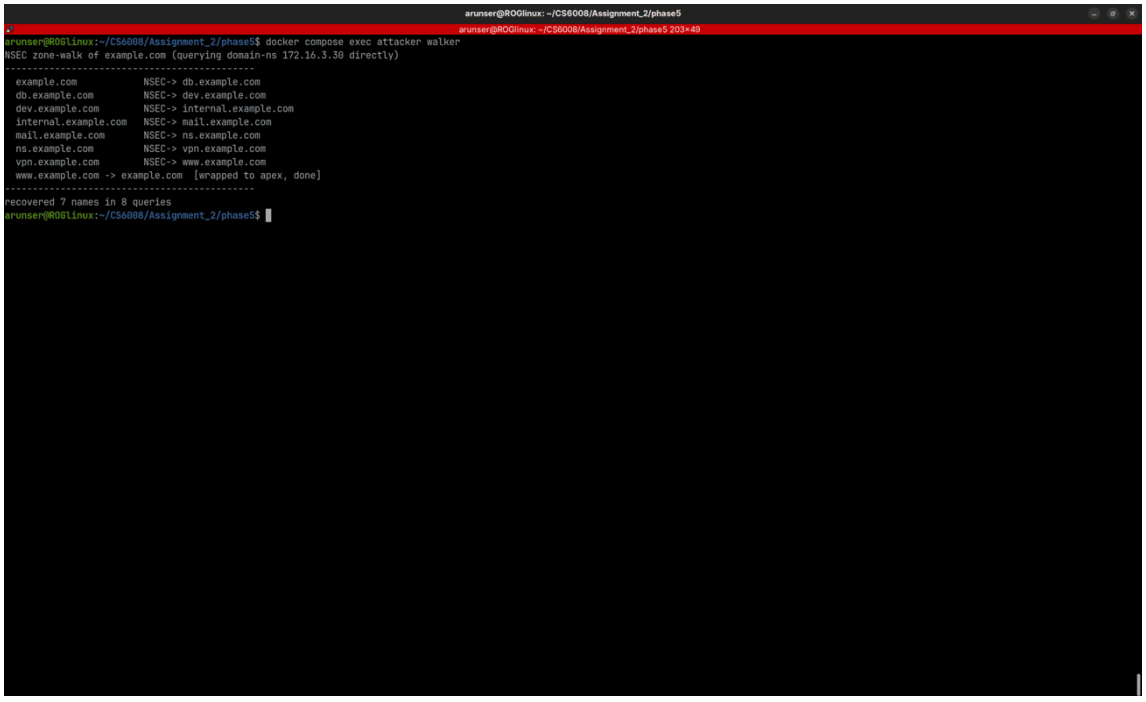


Figure 25: The walker follows the NSEC chain and recovers all seven names in eight queries.

The configuration that made this possible is plain NSEC. An authenticated denial names the next existing owner to prove that the gap between two names is empty, so those records chain into a sorted list of every name in the zone. Plain DNS does not leak this way, because its NXDOMAIN proves nothing in advance and names no neighbour. The signing is what gives the attacker a verifiable map of the zone.

The fix is NSEC3, which hashes the names before chaining them, so an online walk yields only hashes. We re-signed with NSEC3 and ran the walker again. The denial still authenticates, coming back as NXDOMAIN with the authenticated data flag, but the walker now has nothing to follow and recovers zero names.

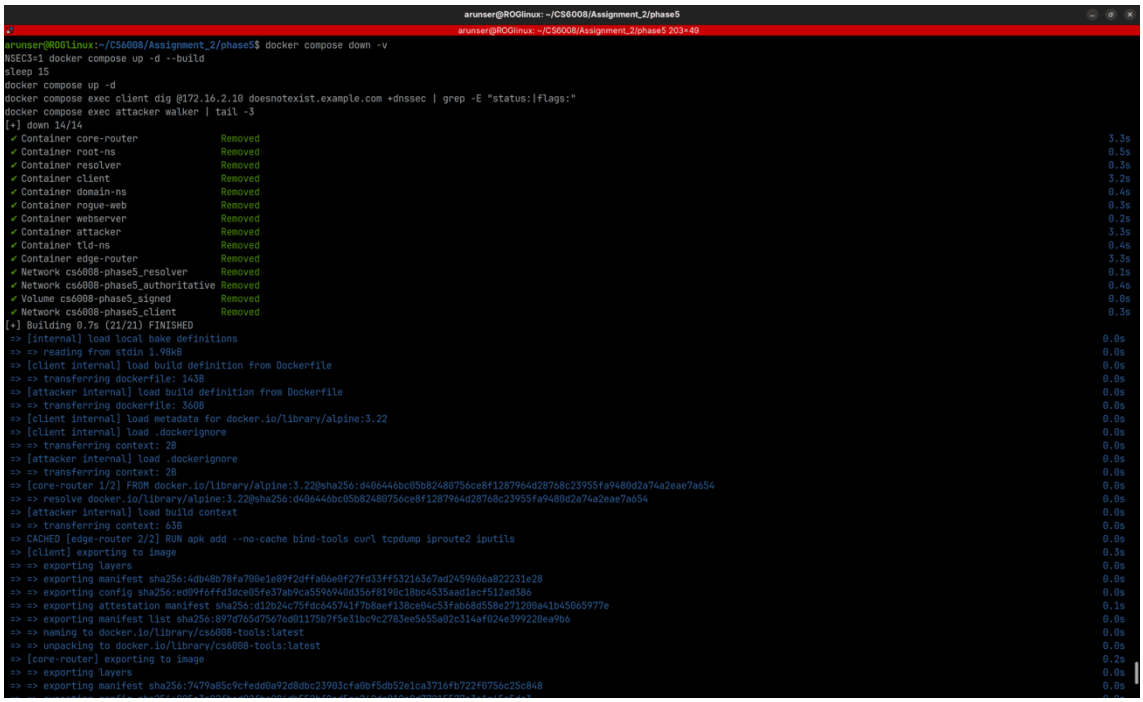


Figure 26: Re-signing the zone with NSEC3.

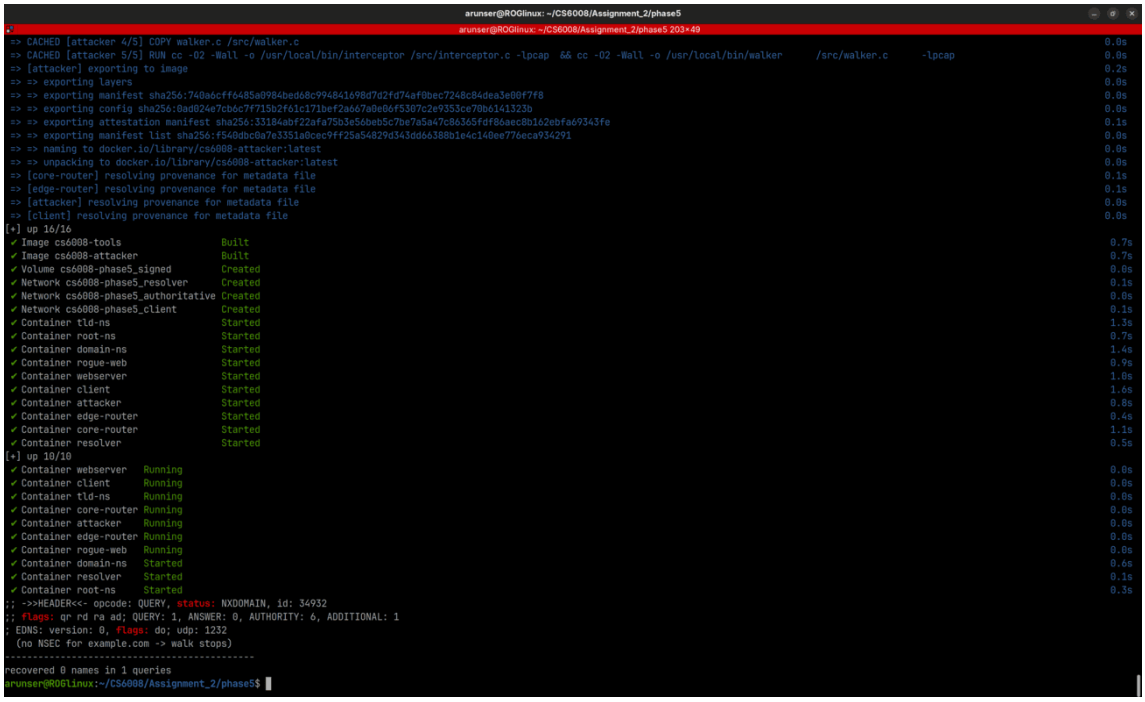


Figure 27: Under NSEC3 the denial still validates but the walk stops and recovers nothing.

The fix is not free. It costs the operator extra signing and validation work and larger responses. And it does not close enumeration completely, because an attacker can still harvest the NSEC3 hashes and crack them offline against a wordlist of likely names. NSEC3 raises the bar rather than removing the problem.

7 Phase 6: The last mile

The resolver validates everything, so the forgeries from the earlier phases die at the resolver. But validation at the resolver only protects the resolver. The hop from the resolver to the client is still plain unsigned DNS, and a stub client takes whatever arrives on it. In Phase 6 the attacker moves onto the client subnet and forges that last hop. We ran two clients from the same machine, a plain stub that just trusts the resolver's answer, and a locally validating client that revalidates the chain itself against the root anchor. First we confirmed the transparent baseline returns the real page.

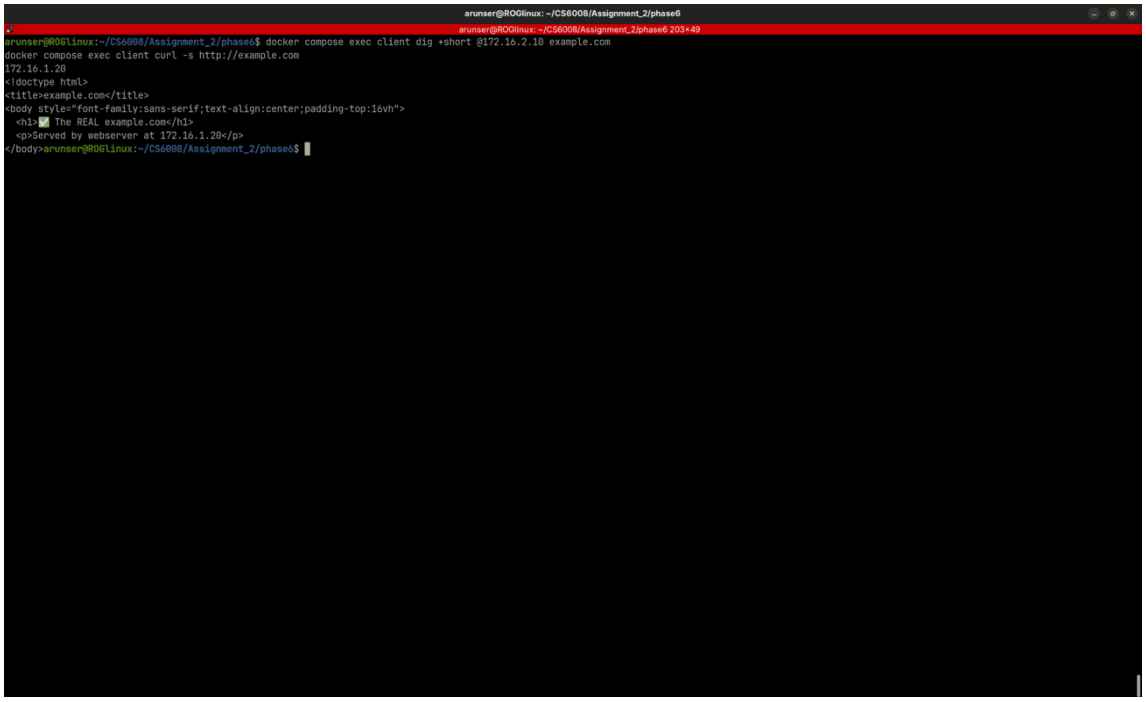


Figure 28: With forging off, the client gets the real answer and the real page.

With forging on, the plain stub is fooled onto `172.16.1.30` and the rogue page, while the validating client refuses the answer and reports that the signature failed to verify.

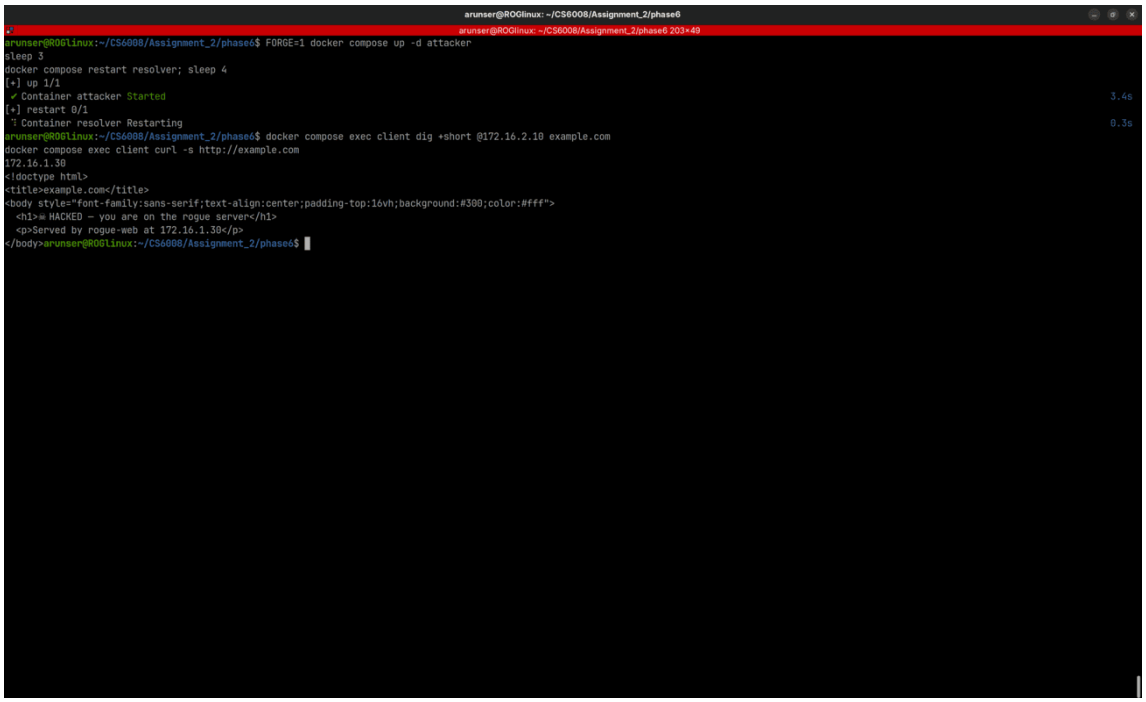


Figure 29: The stub client is fooled onto the rogue server.

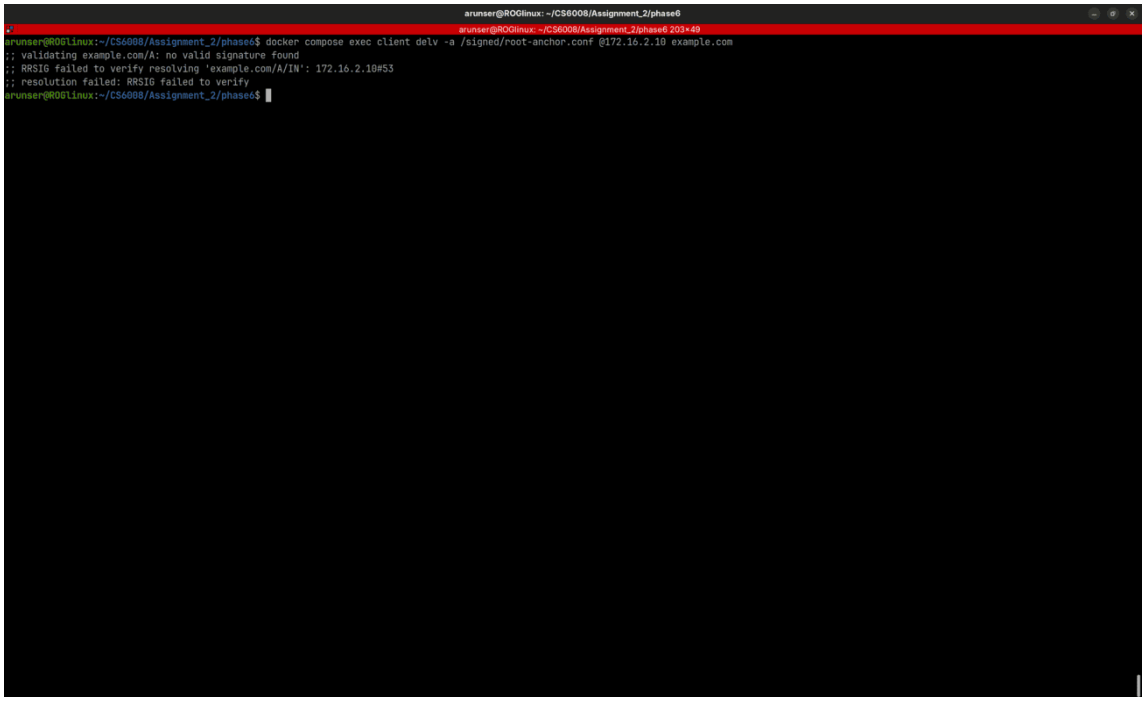


Figure 30: The locally validating client revalidates the chain, finds no valid signature, and refuses the forged answer.

The assignment asks why upstream DNSSEC does not stop this. Validation happens at the resolver, but the stub places its trust in the resolver, and the link between them is plain DNS that the stub cannot verify. Where validation happens and where trust is placed are two different things, and here the stub trusts a hop it has no way to check. It also asks about the authenticated data flag. That flag tells a stub that the resolver validated the answer, but it is just a bit in a cleartext packet, so an on-path attacker on the last mile can set it at will. It is a claim, not a proof. The validating client does not rely on it. It fetches the signatures and revalidates the whole chain from the root anchor on its own, which is how it catches the forged, unsigned address. The takeaway is that a stub is only as safe as the hop to its resolver, and securing that hop is the subject of the final phase.

8 Phase 7: DNS over TLS

Phase 7 closes the last mile. The client now talks to the resolver over TLS on port 853 and validates the resolver's certificate against a private certificate authority that we stand up with OpenSSL. We first checked the healthy case. The client resolves over a TLS 1.3 session, gets `172.16.1.20` with the authenticated data flag, and a capture of the cleartext DNS port during that lookup shows zero packets, so the last mile is now encrypted as well as authenticated.

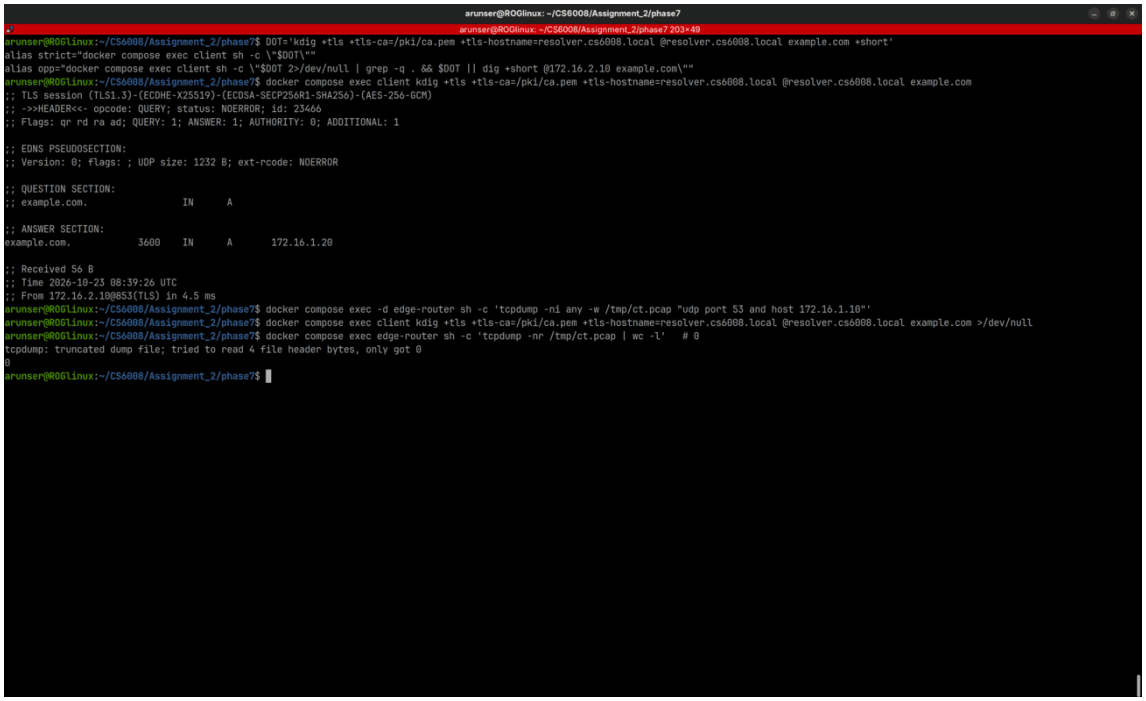


Figure 31: A lookup over DNS over TLS, with no cleartext DNS on the wire for that query.

We then tried three attacks. In the first the attacker forges the cleartext hop exactly as in Phase 6. Both the strict and the opportunistic client stay on the real answer, because the client is on TLS and the cleartext forgery never reaches it.

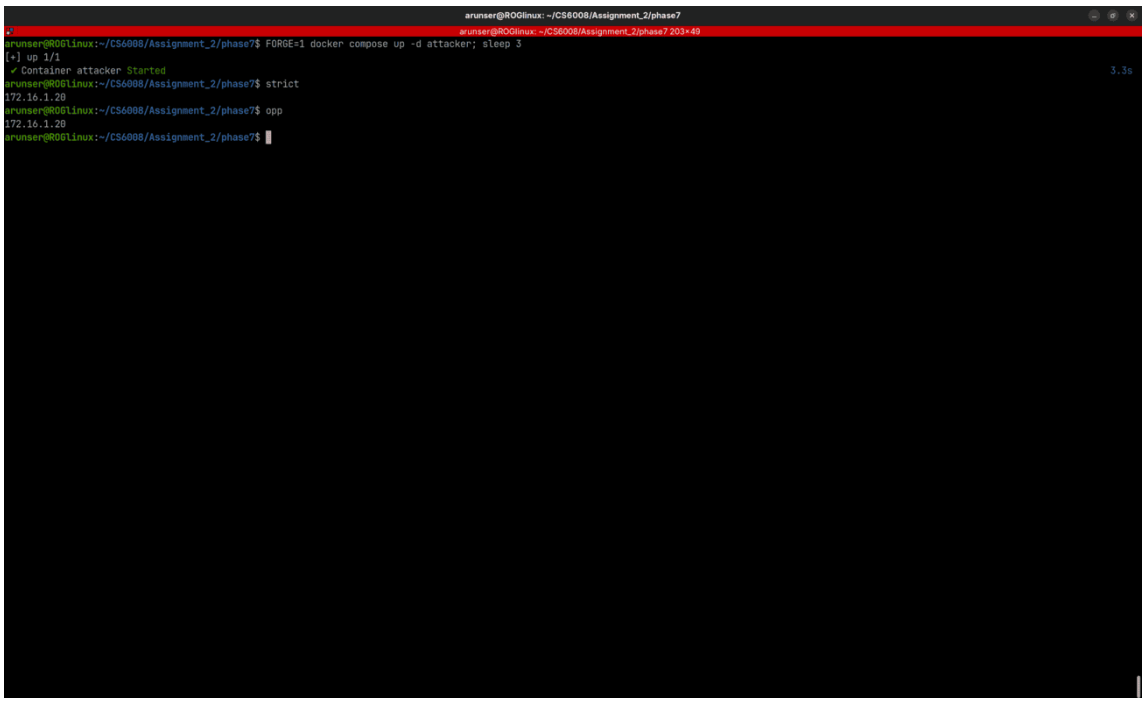


Figure 32: Forging the cleartext hop does not touch the TLS protected client.

In the second attack we block port 853. The strict client fails closed and gets nothing. The opportunistic client, which is willing to fall back to cleartext when TLS is unavailable, falls back and is immediately forged onto `172.16.1.30`. This is the real weakness of opportunistic DNS over TLS, that an attacker who can block the port can strip the protection off a client that is willing to downgrade.

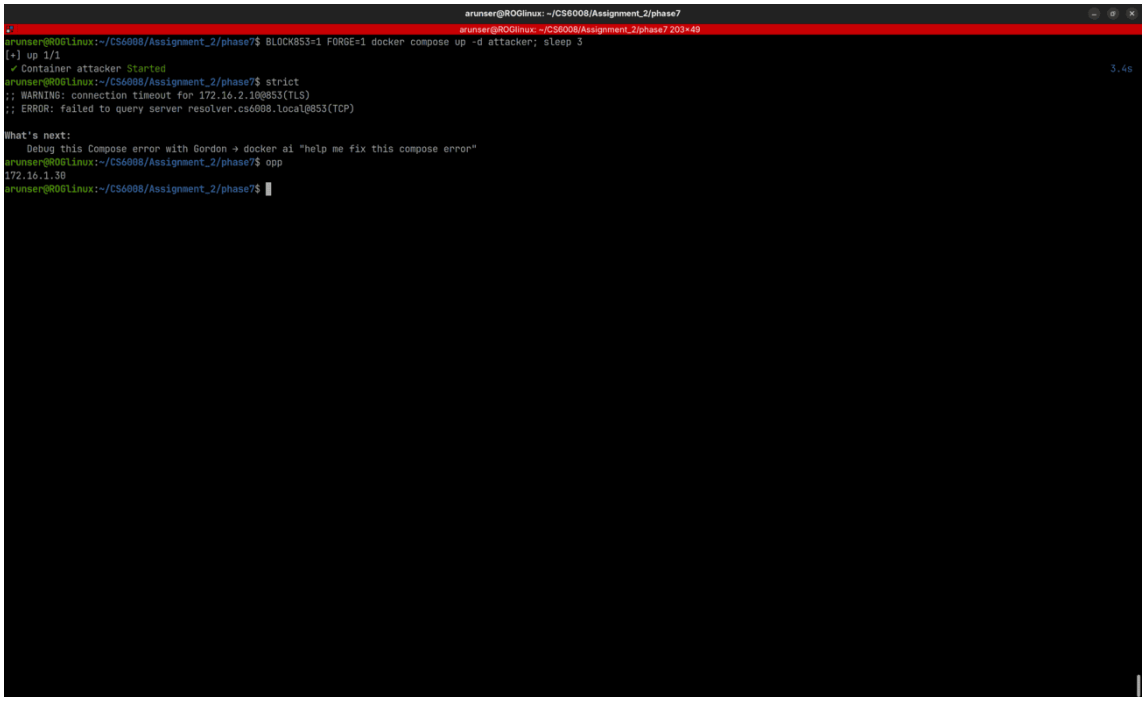


Figure 33: Blocking port 853 downgrades the opportunistic client back to forged cleartext.

In the third attack the edge router redirects the client's 853 traffic to a rogue DNS over TLS server. The rogue holds a certificate for the resolver's name, but it is not signed by our certificate authority. The strict client's handshake therefore fails with a certificate error, because the client reached the rogue server and rejected its certificate. The opportunistic client again falls back to cleartext and is forged. The certificate authority is what stops the impersonation here, in exactly the way a public authority would on the real internet.

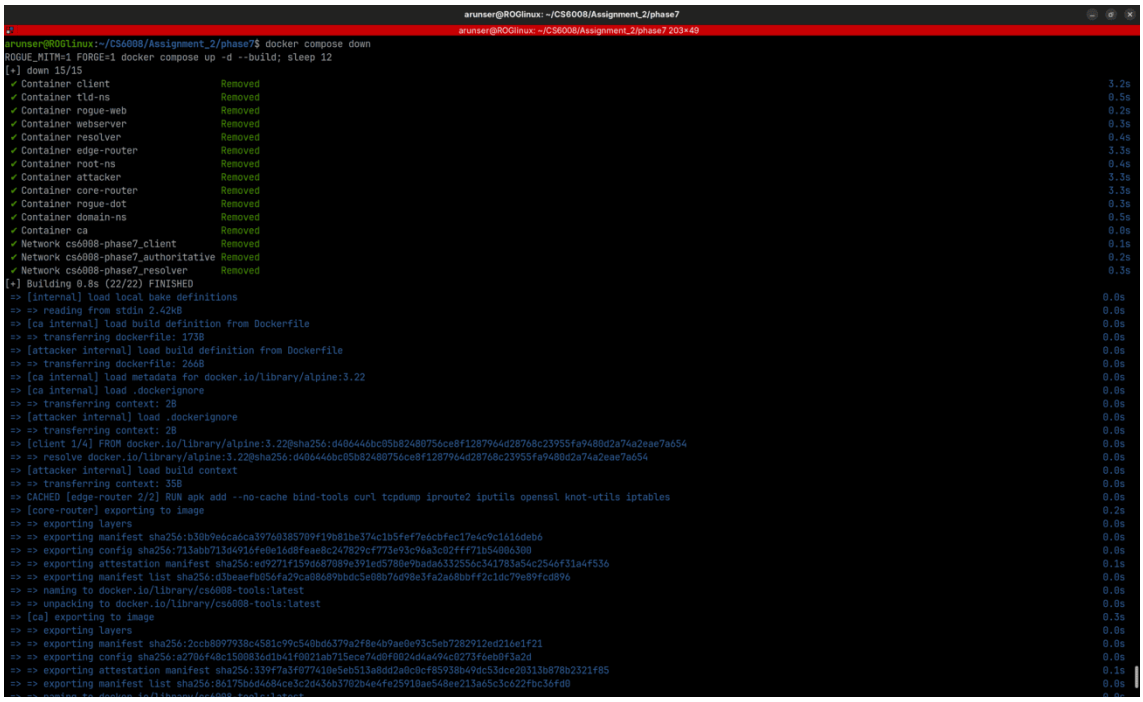


Figure 34: Deploying the rogue DNS over TLS server and the redirection.

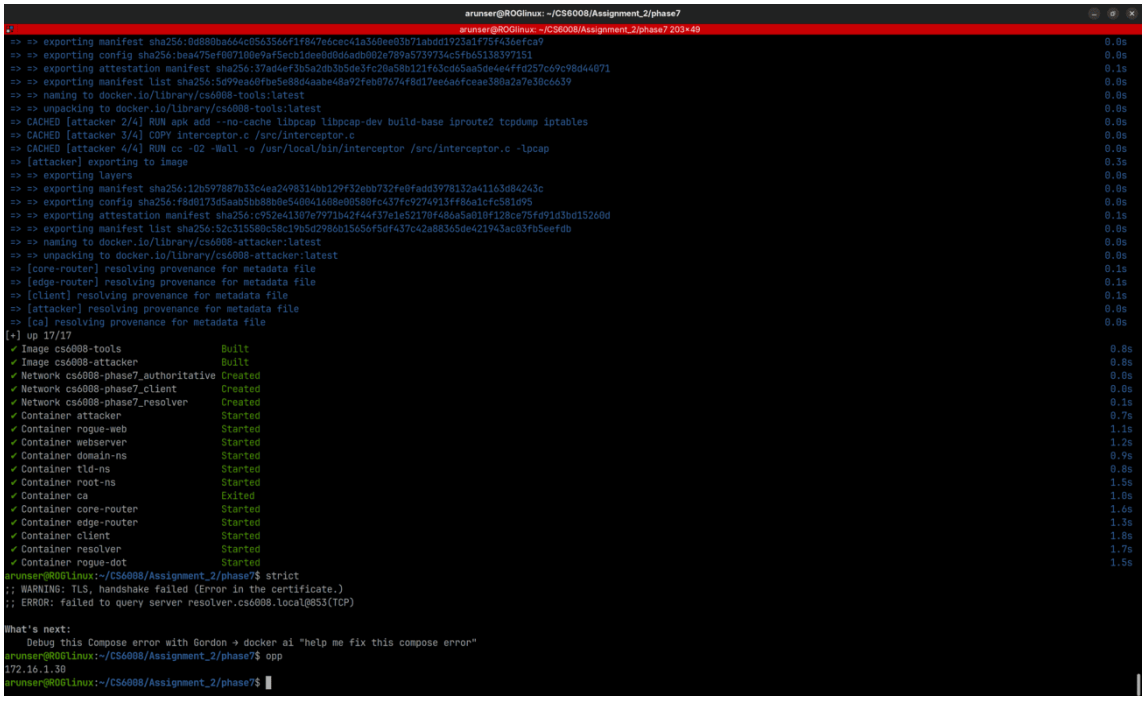


Figure 35: The strict client reaches the rogue server and refuses its untrusted certificate, while the opportunistic client falls back and is forged.

The assignment asks which of DNS over TLS and DNS over HTTPS we deployed and its drawbacks. We deployed DNS over TLS. Its main drawback against DNS over HTTPS is that it uses a dedicated

and well known port, which is trivial to identify and block at a firewall, which is exactly the downgrade we demonstrated. DNS over HTTPS hides inside ordinary HTTPS on port 443 and is much harder to single out and block, at the cost of more complexity and of mixing its traffic in with the web.

Finally, it asks what an encrypted and authenticated last mile still does not protect. It protects the hop between the client and this resolver, but the resolver still sees every query in the clear and has to be trusted. Traffic metadata such as timing, sizes and the addresses the client goes on to contact still leaks. And it does nothing for the path from the resolver to the authoritative servers, or for a resolver that is itself malicious or compromised. Confidentiality of the first hop is not the same as anonymity or as end to end integrity.

9 Reflections

Taken together the seven phases trace a single argument. Plain DNS trusts whatever arrives, and on a shared path that is enough for an on-path attacker to redirect a name at will. DNSSEC fixes integrity by signing the data, and it holds up even when the attacker tries to strip the signatures off, as long as the parent actually publishes the delegation signer that vouches for the child. The same signing then creates a new problem, zone enumeration, which NSEC3 reduces but does not remove. None of this protects a stub on the last hop, and none of it hides the queries, which is why DNS over TLS and a certificate authority are needed to encrypt and authenticate the client to resolver link. Each protection is narrow and solves exactly one thing, and the exercise of attacking the previous phase before trusting the next one is what made the boundaries clear.

10 Use of generative AI

We used an AI assistant throughout this assignment as a tutor. We asked it to explain concepts and review our work rather than to write the solution for us, and we ran, captured and checked everything ourselves. The prompts fell into the following groups.

Environment and tooling. Questions about laying out the three subnets and the two routers in Docker Compose, how to add static routes and disable NAT so the only path between subnets is through the routers, how to run BIND under the right user inside the container, and how to capture the right traffic with tcpdump and Wireshark on the correct interface.

Concepts. Explanations of how recursive resolution walks the root, TLD and domain referral; why plain DNS has no integrity and what an on-path position buys over an off-path one; how the DNSSEC chain of trust runs from a root anchor through the delegation signer in each parent; why stripping signatures fails when the parent vouches for the child and why a missing delegation signer reopens the attack; how plain NSEC leaks a zone and how NSEC3 reduces that; the difference between where validation happens and where trust is placed on the last mile; what the authenticated data flag really means; and the trade-offs between DNS over TLS and DNS over HTTPS.

Implementation help. Walkthroughs of libpcap for capturing and injecting packets, why the IP and UDP checksums have to be recomputed after editing a response, how to parse DNS names and records on the wire, how to sign zones and build the delegation chain with the BIND tools, and how to create a private certificate authority and server certificates with OpenSSL.

Debugging. Asking why the resolver dropped our re-injected packets until we recomputed the

checksums, why the attacker lost the resolver's address after a restart until we re-learned its MAC, why appending a delegation signer silently broke the chain until we fixed a missing newline, why the TLS listener needed the keywords in a particular order, and why the rogue DNS over TLS redirection did nothing until we added iptables to the router image.

Review and evidence. Checking each phase's screenshots against the requirements and confirming what each one demonstrates, and help structuring this report.